2 – DATATYPES

When learning a new programming language, one of the first things to understand is the set of **native (primitive) data-type** it supports. A solid grasp of data types is essential for writing **robust**, **readable** and **efficient code**, and it helps you become more fluent in the language.

Unlike many other statically typed languages you may have studied (such as C# or Java), **Python is a dynamically** typed. Other common dynamically-typed languages include Ruby and JavaScript™). This means that a variable is **not bound to a single data type** and may refer to different types overthe life time of a program.

> FYI
>
> A statically typed language is a programming language where variable types are checked at compile-time (before the program runs), rather than at runtime.

In this chapter, we focus on Python's primitive (basic data) types including:

- `int`
- `float`
- `bool`
- `str`
- None

Non-primitive (composite data) types such as **`set`**, **`list`**, **`tuple`** and **`dict`** are introduced here and explored in more detail in later chapters.

You can infer the type of most literals by inspection:

- `str`: anything enclosed by single, double or triple quotation
- `int`: series of digits
- `float`: series of digits with a period
- `bool`: True or False
- `set`: items separated by commas and enclosed by a pair of curly braces
- `tuple`: items separated by commas and enclosed by a pair of parentheses
- `list`: items separated by commas and enclosed by a pair of square brackets
- `dict`: key-value pairs separated by commas and enclosed by a pair of curly braces

2.a – None

None is a special constant in Python that represents the absence of a value. It is **not equal to anything** except to another None. This is similar to the Null object in C#.

```
>>> None == 0
False

>>> None == []
False

>>> None == False
False

>>> x = None
>>> y = None
>>> x == y
True
```

None is its own object with its own type:

```
>>> type(None)
<class 'NoneType'>
```

2.b – Numerics

Numbers object in Python are **immutable**, meaning their values cannot be changed in place. When a new value is assigned, Python creates a new object in memory.

There are three types of numbers available without any imports:

2.b.1 – Integers

Integers are whole numbers with unlimited precision. Python support integer literals in several bases:

Fun fact
C# has 13 primitive numeric types

- Base 10 (decimal) – default
- Base 2 (binary) – prefix `0b`
- Base 8 (octal) – prefix `0o`
- Base 16 (hexadecimal) – prefix `0x`

Some int literals

```
>>> 7                    #base 10 literal
7

>>> 0b100                #base 2 literal (prefix 0b)
4

>>> 0o27                 #base 8 literal (prefix 0o)
23

>>> 0xf                  #base 16 literal (prefix 0x)
15
```

Python also allows underscores as **digit separators** to improve readability.

```
>>> 0b100_0000           #binary literal with digit separator
64

>>> 100_000              #decimal literal with digit separator
100000
```

Integer can grow arbitrarily large:

```
>>> 12345678901234567890123456789012345678901234567890 + 1
12345678901234567890123456789012345678901234567891
```

2.b.1.2 – bool

This is an int sub-type, having only two literal bool values: **True** (1) or **False** (0)

> Fun fact
>
> The objects **True, False** and **None** are considered immortal in python.

```
.
>>> False + 0
0

>>> True + 0
1

>>> True + True
2
```

Unlike C#, every python object has a truth value. The following are considered `False`:

- None,
- `False`
- Zero numeric type: `0`, `0l`, `0.0`, `0j`
- Any empty sequence: `''`, `{}`, `[]`

All other values are considered `True`

```
>>> bool([])           #empty list
False
>>> bool([0])
True
>>> bool('')           #empty string
False
>>> bool('0')
True
```

```
>>> bool(0)           #integer zero
False
>>> bool(3.1415)
True
```

2.b.2 – Floating Point Numbers

Floats represent number with decimal places. Internally, they are stored as a **binary fraction**, which means not all decimal values can be represented exactly.

> Fun fact
>
> Most languages are not able to represent float value exactly. The Decimal type in the decimal module is able to represent a decimal number exactly.

```
>>> .1 + .2
0.30000000000000004
```

Any division produces a float.

```
>>> type(4 / 2)
<class 'float'>
```

Python has **only one floating-point type**, unlike C#, python only has a single float type.

You should avoid using float for high precision arithmetic as required for financial applications. Use the decimal module instead.

```
10., .001, 1e100, 3.14e-10
3.141_592                           #approximate value of pi
5.972e24                            #mass of the earth
9.1093837e-31                       #mass of an electron in kg
```

2.b.3 – Complex Numbers

Complex numbers consist of a **real** and an **imaginary** part. In Python a complex literal must include an imaginary component, indicated by `j`. The following are some examples of complex literals:

> Fun fact
>
> The Italian mathematician **Gerolamo Cardano** first used complex number in the 16th century.

```
3.14j, 10.j, 10j, .001j, 1e100j, 3.14e-10j
1.414_213j                          #root of -2
```

2.b.4 – Fraction (informational)

This section is for information only. The fraction datatype is available via an explicit import only. You are able to used numbers as ratios of integers.

2.b.5 – Decimal (informational)

This section is also for information only. The decimal datatype is available via an explicit import only.

2.c – Sequences:

Python provides several sequence types, which shares some common behaviors such as being **iterable** and **indexable**:

- `str`
- `tuples` -> ()
- `lists` -> []
- `range` -> range(5)

`stack` and `queue` will not be covered in this course

2.c.1 – Strings

Strings are immutable sequences of Unicode characters. It is delimited by matching quotes:

- Single quotes `'Hello'`
- Double quotes `"Hello"`
- Triple quotes. `'''Hello'''`

In this course, we will use a single quote for strings with a few exceptions. Triple quotes (either single or double) are used for multi-line string such as docstring.

```
a = 'Hello '
b = "world"
c = '''This string
can span multiple
lines.'''
```

2.c.1.1 – F-Strings

f-strings allow variables and expressions to be embedded directly in string.

```
days = 365
print(f'There are {days} in a year')
```

2.c.2 – Tuple

Tuples are immutable sequences. Once created, their contents cannot be changed. The items do not have to be the same type.

```
>>> a = ()                          #verbatim definition
>>> a
()

>>> b = (1)                         #this will not do a single item tuple
>>> b
1

>>> b = (1, )                       #now you have a single item tuple
>>> b
(1,)
```

2.c.3 – Range

range represents a sequence of integers commonly used in loops.

```
>>> range(4)
range(0, 5)
```

2.c.4 – List

Lists are mutable sequences and are the most commonly used collection type in Python. It serves the purpose of arrays in other languages and it may contain any type of members. The following statements will return lists:

```
>> a = []                          #verbatim definition
>> a
[]

>> b = 'Hello world'.split()       #list from calling split on a string
>> b
['Hello', 'world']
```

2.d – Mappings:

2.d.1 – Dictionary

A dictionary stores **key-value pairs**.

- Key must be **unique**, **hashable** and **immutable** such as strings, numbers, or tuples. A list may not be used as a key.
- Values may be any python object
- Dictionaries does **not** support positional indexing, only key-indexing.

This data type is flexible and powerful and is extensively used in the language and will be heavily used in this course.

```
#there are lots of ways of creating a dictionary
>>> a = {'red': 'Danger', 1: 'first', 'second': 2, ('a', 'set'): 'a tuple
is my key'}
```

```
>>> b = dict(zip(['ilia', 'hao', 'yin'], ['Ilia Nika', 'Hao Lac', 'Yin
Li']))


>>> c = dict([('arben', 'Arben Tapia'),('patrick','Patrick Gignac')])


>>> d = dict(nar='Narendra Pershad', joanne='Joanne Filotti')
```

2.e – Set types

Sets unordered collection of unique elements. They are iterable but not indexable

Common uses:

- Fast membership testing
- Removing duplicates from a sequence
- Mathematical operations such as intersection, union, difference and symmetric difference.

Set (mutable) and Frozen sets (immutable).

2.e.1 – Set

A sequence of unique immutable python objects. Sets, list or dict cannot be in a set. Set does not support indexing.

```
>>> {1, 2, 3}                    #immutable sequence of bytes
{1, 2, 3}
```

Mimics a Mathematical set such that no duplicates are allowed.

Only immutable object may be elements of a set.

2.f – Binary types

These types are used to handle binary data. They include the following:

```
>>> a = b'ABC'                   #immutable sequence of bytes
>>> print(a)
b'ABC'
>>> b = bytearray([65, 66, 67])  #mutable sequence of bytes
>>> print(b)
```

```
bytearray(b'ABC')
```

2.g – Enum

This datatype is available via an explicit import. We will not be using this type in Programming 1.

2.h – Checking Data Types

Use the `type()` function to inspect the data type of a value.

```
>>> type('Hello')
<class 'str'>

>>> type(3)
<class 'int'>

>>> type(True)
<class 'bool'>

>>> type(3.141592)
<class 'float'>

>>> type([])
<class 'list'>

>>> type(())
<class 'tuple'>

>>> type({1,})
<class 'set'>

>>> type({})
<class 'dict'>
```

2.i – Converting Data Types

Most languages provide a mechanism to convert from one data type to another type either through implicit or explicit casting You can convert a variable from one data type to another.

2.i.1 – Implicit

This kind of conversion is automatic e.g.:

```
a = 3.15                        #this is a float
b = 5                           #this is an int
c = a + b                       #b is up cast to a float before addition


#the following does not work (it does work in C#)
a = 'Hello'                     #this is a str
b = 5                           #this is an int
c = a + b                       #this does not work
```

2.i.2 – Explicit

Explicit casting is done by constructor methods such as int(), float() and str():

2.i.2.1 – Converting to string:

```
str()
n = 354
print(str(new_n))         #result 354
```

2.i.2.2 – Converting to int:

```
>>> int(3.14)             #truncates down the float
3
>>> int(-1)
-1
>>> int(0b110)            #binary literal
```

```
6
>>> int('110', base=2)
6
>>> int('fe', base=16)
254
>>> int('11', base=12)
13
```

This converts a number or string to an integer assuming base 10. If you would like to specify an alternate base, then the argument must be a string. The string literal must be a legal string: 'one' is not a legal number and will cause a `ValueError` to be raised.

2.i.2.3 – Converting to float:

```
>>> float(-1)
-1.0
>>> float('3.1415')
3.1415
>>> float(1e8)
100000000.0
>>> float(1.2e-3)
0.0012
```

Other conversions can be done with: `tuple()`, `list()`, `set()`, `dict()`, `ord()`, `chr()`, `hex()`, `oct()`, `bin()`

2.j – Operators

Operators usually takes two operands to create expressions. This expression is evaluated to gives a result. The operator determines the type of operands as well as the resulting value. Unary operators only require a single operand whilst ternary operators require three operands.

2.j.1 – Assignment Operators

This operator binds a value to a memory location (name). i.e it sets the value of a variable. The left-hand side value of the operator is bound to the variable on the right-hand side. If the left side is an expression, then it is evaluated and the resulting value is set to the variable on the right-hand side.

```
>>> a = 5           #a is 5

>>> a = 4 * 3       #a is 12

>>> a *= 3          #a is 3 times a or 36

>>> a = b = c = 0   #a, b, and c is set to 0

>>> a, b, c = 0, 1, 2     #a is 0, b is 1 and c is 2

>>> a, b = b, a     #values of a and b are swapped

>>> a = [0, 1, 2, 3, 4]
>>> b, *c = a
>>> b
0
>>> c
[1, 2, 3, 4]


>>> *b, c = a
>>> b
[0, 1, 2, 3]
>>> c
4
```

2.j.1 – Arithmetic Operators

You can do mathematical calculations with Python as long as you know the right operators. The order of precedence of the operator is listed in order.

```
>>> 5 ** 2          #exponent
25

>>> 4 * 3           #multiply
12

>>> 18 / 7          #divide
2.5714285714285716

>>> 18 // 7         #integer divide
2

>>> 18 % 7          #modulus
4

>>> 2 + 7           #add
9

>>> 9 - 4           #subtract
5
```

When working with different numeric types, Python will up-convert the lower precision value to a float and then perform the computation. Often the result of such arithmetic is a float.

2.j.1.1 – The Modulo Operator

Often, you'll want to know what is the remainder after a division. e.g. 4 ÷ 2 = 2 with no remainder but 5 ÷ 2 = 2 with 1 remainder The modulo does not give you the result of the division, just the remainder. It can be really helpful in certain situations, e.g. figuring out if a number is odd or even.

```
5 % 2              #result is 1
```

2.j.2 – Relational (Comparison) Operators

These operators are normally used to compare two values of the same type. The result is a boolean value. You may also use these operators with string type values.

```
>>> 3 == 4          #equal
False

>>> 3 != 4          #not equal
True

>>> 3 > 4           #greater than
False

>>> 3 < 4           #less than
True

>>> 5 >= 3          #greater than or equal to
True

>>> 5 <= 3          #less than or equal to
False
```

2.j.3 – Logical Operators

Use to compare two values normally of the same type. The result is a bool. You may also use these operators with string type values.

```
>>> True and True   #True only if both operands are True
True

>>> True and False
False

>>> True or True
True

>>> True or False
True

>>> False or False  #False only if both operands are False
```

```
False

>>> not True        #inverts the value of the single operand
False

>>> not False
True
```

2.j.3.1 – Short-circuit evaluation:

If the value of an expression can be computed without evaluated all the part, then the parts are not evaluated.

```
>>> x = 0
>>> y = 5
>>> (y != 0) and (x /y > 0)            # division never happens
```

2.j.4 – Identity Operators

Identity operators are used to compare the objects, not if they are equal, but if they are actually the same object, with the same memory location.

```
>>> a = [3]
>>> b = [3]
>>> a == b          #compares the values
True

>>> a is b          #compares the memory addresses
False

>>> id(a)
2354776320832

>>> id(b)
2354776273920
```

2.j.5 – Membership Operators

This can be used to test if a value is present in a sequence (string, list, tuple or dict).

```
>>> 'a' in 'Narendra Pershad'
True

>>> 'q' in 'Narendra Pershad'
False

>>> 'hello' in ['hello', 'world']
True

>>> 1 in {'one': 1, 'two': 2, 'three': 3}
False

>>> 1 not in {'one': 1, 'two': 2, 'three': 3}
True

>>> 'one' in {'one': 1, 'two': 2, 'three': 3} #for dict, the in operator
only check the keys
True
```

2.j.6 – Bitwise Operators

This category of operators requires two integer operands (except ~ not) and it returns an integer. It operates at the bit-level, so the results are not immediately understandable. However, examining the results (and operands) as binary value will make the operators more palatable.

```
>>> 9 << 2
36

>>> print(f'{bin(0b01001 << 2)}')
0b100100

>>> print(f'{bin(0b01001 << 3)}')
0b1001000
```

```
>>> print(f'{bin(0b01001 >> 2)}')
0b10

>>> print(f'{bin(0b01001 & 0b00101)}')
0b1

>>> print(f'{bin(0b01001 | 0b00101)}')
0b1101

>>> print(f'{bin(0b01001 ^ 0b00101)}')
0b1100

>>> print(f'{bin(~0b01001)}')
-0b1010
```

2.j.6.1 – The augmented Operator

Augmented assignment provides a short way to perform a binary operation (arithmetic, boolean or bitwise) and assigning the results back to one of the operands. You can also use any of the other mathematical operators e.g. -=, +=, *=, /=, //=, **=, |=, >>=, <<=, %=, or ^=

```
my_number = 4
my_number += 2      #my_number is now is 6
```

2.j.7 – Operator precedence

If you have an expression with more than one operator, then python uses a set of internal rules to decide the order that the computation must follow. The table below list the various operators in descending order of precedence.

Operator	Description
()	Parentheses
**	Exponentiation

+x -x ~x	Unary plus, unary minus, and bitwise NOT
*** / // %**	Multiplication, division, floor division, and modulus
+ -	Addition and subtraction
<< >>	Bitwise left and right shifts
&	Bitwise AND
^	Bitwise XOR
|	Bitwise OR
== != > >= < <= is is not in not in	Comparisons, identity, and membership operators
not	Logical NOT
and	AND
or	OR

2.k – Summary

This chapter introduces Python's core data types and how they behave. Python is a **dynamically typed** language, meaning variables are not locked to a single data type and can change during a program's execution.

You learn about **primitive data types**, including None, numeric types (`int`, `float`, `bool`, `complex`), and strings. None represents the absence of a value, numeric types are immutable, integers have unlimited precision, floats represent decimal values with limited precision, and `bool` is a subtype of `int`. Python evaluates all objects as either *truthy* or *falsy*, which is important for conditionals.

The chapter also introduces **composite data types** such as sequences (`str`, `list`, `tuple`, `range`), mappings (`dict`), and sets. Sequences are iterable and indexable, with Python supporting negative indexing. Lists are mutable, tuples are immutable, dictionaries store key–value pairs, and sets hold unique, unordered elements.

You explore **type checking** using `type()` and **type conversion**, both implicit and explicit, using constructors like int(), `float()`, and `str()`.

Finally, the chapter covers **operators**, including assignment, arithmetic, comparison, logical, identity, membership, and bitwise operators, along with operator precedence. Understanding these concepts forms the foundation for writing correct, readable, and efficient Python programs.